

SESAP — a support desk where one click sets a real process in motion

SESAP (Stanbic IBTC Enterprise Support & Automation Platform) is a UiPath Coded App backed by Data Fabric, orchestrated by a Maestro BPMN process, and executed by a published RPA robot. This document explains what each layer does, exactly how they connect, how to run it, and which bonuses were attempted.

Coded App

→

Maestro (BPMN)

→

RPA robot

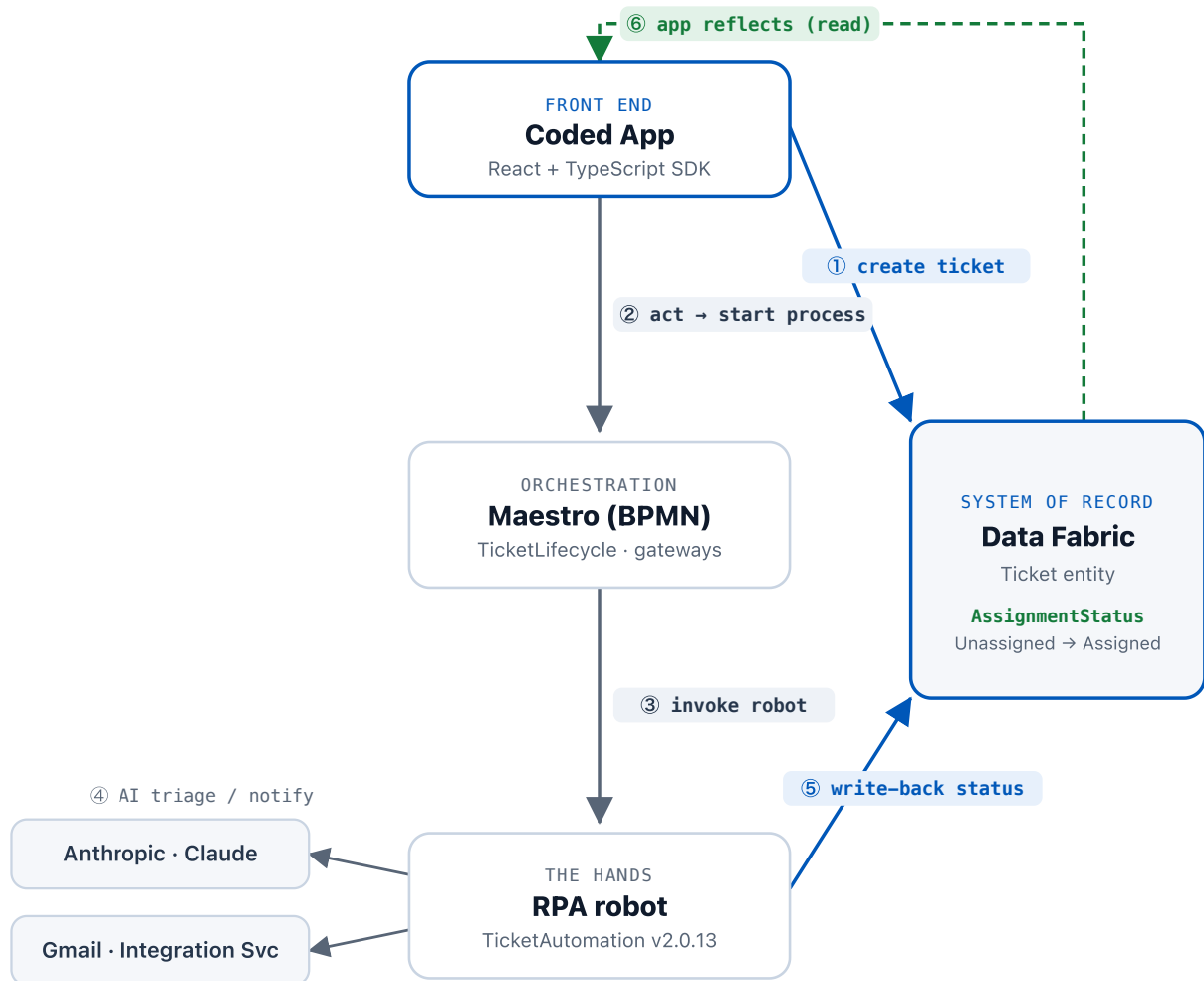
→

Data Fabric

↗ APP REFLECTS THE CHANGE

Architecture at a glance

One action in the app travels through orchestration and automation, writes to the system of record, and returns to the app as a visible change. The numbered arrows are the same six hops described below.



①② app-initiated ③ Maestro invokes the robot ④ robot calls AI + email ⑤ robot writes back

⑥ app reads & reflects — loop closed

The four layers

Each layer has one job. The detail below is what a judge can verify in the source.

FRONT END

Coded App

A React/TypeScript operational dashboard support leads use to view every active ticket, search and sort, open a full ticket record, create tickets, and act on them. It reads and writes Data Fabric directly and starts the orchestration. Three role-based experiences (supervisor / agent / customer) come from one codebase.

Vite build · @uipath/uipath-typescript
SDK · deployed as a Coded Web App
(uipath.host) · platform OAuth
(DataFabric + OR scopes)

SYSTEM OF RECORD

Data Fabric

Holds the `Ticket` entity (24 fields — 19 business, 5 system). Both the app and the robot read and update it.

`AssignmentStatus` is the field the automation flips to prove the write-back loop; `UpdateTime` gives real resolution-time analytics.

Entity id ca14332a... · SDK
entities.getAllRecords / update ·
schema in docs/DATA_FABRIC_SCHEMA.md

ORCHESTRATION BRAIN

Maestro (BPMN)

A BPMN process, `TicketLifecycle`, that models the ticket lifecycle: a start event, an *Action* gateway routing submit / resolve / close / escalate, triage and routing tasks, a risk gateway that sends high-risk tickets to manual assignment, and end events. It invokes the robot as a service task.

Deployed Maestro release · start +
gateways + tasks + ends · passes
ticket context as input args

THE HANDS

RPA robot

A published automation,
`TicketAutomation`, that does the concrete work: AI triage (categorise / prioritise / summarise / recommend), auto-assignment to the right team, the Data Fabric write-back, and branded lifecycle emails (CC the supervisor).

SESAP.process.TicketAutomation v2.0.13
· Data Service + Anthropic
(InvokeCode) + Gmail activities ·
reads the key from an Orchestrator
asset

How the layers connect

The six hops from the diagram, with the actual mechanism behind each.

1

App → Data Fabric

Creating or editing a ticket writes it straight to the `Ticket` entity as `Unassigned`.

TypeScript SDK `entities` API using the Coded App's platform auth.
Field mapping lives in `toEntityRecord`; failures never block the UI.

2

App → Maestro

Acting on a ticket — create, resolve, close, escalate — starts the Maestro process for that specific action.

`processes.start` passes the action + ticket context (Ref, Subject, Category, RiskLevel...) as input arguments, which drive the BPMN gateways. A "Live automation" toggle gates this to conserve Robot Units during development.

3

Maestro → RPA

The BPMN triages and routes the ticket, then invokes the published robot as one of its steps.

A Maestro service task starts the `TicketAutomation` job in the `SESAP-Live` folder; high-risk tickets branch to a manual-assignment path instead.

4

RPA → AI & email

The robot asks Claude to categorise, prioritise, summarise and recommend, then emails the requester (CC supervisor).

Anthropic call inside an `InvokeCode` activity, keyed by the Orchestrator asset `SESAP_Anthropic_Key` (never in the repo); Gmail via Integration Service.

5

RPA → Data Fabric

The robot assigns the ticket and writes `AssignmentStatus` from `Unassigned` to `Assigned` — the write-back that proves the loop.

Data Service activities update the record; `UpdateTime` advances, which is what the resolution-time KPI reads.

6

Data Fabric → App

The app re-reads the entity and reflects the new status and assignee — trigger to visible result, loop closed.

`getAllRecords` on load/refresh; `fromEntityRecord` maps fields back, resolving the assignee name to an email so "Assigned To Me" works after a reload.

The Ticket data model

The minimum challenge fields, plus the write-back and timing fields that make the loop and the analytics real.

FIELD	TYPE	ROLE
Ref / Id	STRING / UUID	Human ref + Data Fabric primary key
Subject	STRING	Ticket title
Description	STRING	Full detail
Priority	STRING	low · medium · high · urgent (drives SLA)
Status	STRING	open · in progress · resolved · closed
AssignedTo	STRING	Assignee, set by the robot
CreatedAt	DATETIME	App-set creation time (SLA baseline)
AssignmentStatus	STRING	The write-back proof: Unassigned → Assigned
UpdateTime	DATETIME_TZ	Data-Service managed; powers avg resolution time

Setup & run

Reproducible from the repo. The app and the robot deploy on independent pipelines; both are free of Robot Units to build.

1 Coded App — build, then pack → publish → deploy

```
# all three steps are required – deploy alone is a silent no-op,  
# and the version must be bumped or the old CDN revision is reused  
npm run build  
uip codedapp pack ./dist --name "SESAP Support Platform" \  
  --version <NEW> --content-type webapp --main-file index.html  
uip codedapp publish --name "sesapsupportplatform" --version <NEW> --type Web  
uip codedapp deploy --name "SESAP Support Platform" --folder-key <SHARED>
```

Vite injects the app's OAuth scopes/clientId into `dist/index.html` at build time from `uipath.json`.

2 RPA robot — pack → upload → repoint the release

```
uip rpa pack SESAP/TicketAutomation ./out \  
  --package-id SESAP.process.TicketAutomation --package-version <NEW>  
uip or packages upload ./out/SESAP.process.TicketAutomation.<NEW>.nupkg  
uip or processes update-version <RPA_KEY> --package-version <NEW>
```

The Anthropic API key is never committed — the robot reads it at runtime from the Orchestrator asset `SESAP_Anthropic_Key`.

3 Maestro & Data Fabric

`TicketLifecycle` is deployed as a Maestro release in `SESAP-Live`; the `Ticket` entity is provisioned in Data Fabric (schema in `docs/DATA_FABRIC_SCHEMA.md`). The whole solution is kept in sync to Studio Web with `uip solution upload SESAP --force`.

Deployed resources — the real values

Live in organization `stanbgdgzsbd`, tenant `DefaultTenant`.

Live app	<code>https://stanbgdgzsbd.uipath.host/sesap-support-platform · v2.9.19 (rev 32)</code>
Shared folder key	<code>1c313714-4ed8-41a8-91f2-edfa178a3902</code>
SESAP-Live folder	<code>c034c633-bf82-47ba-9e35-8955ea38d2b0</code>
Ticket entity	<code>ca14332a-0081-f111-b338-000d3ab4d3b7 · 24 fields</code>
RPA release	<code>SESAP.process.TicketAutomation v2.0.13 · 38E6499B-5D0E-48BF-8456-65B37D6689B6</code>
Maestro process	<code>TicketLifecycle · fedfe5d6-49f8-4cab-9503-d11f2d8ddb81</code>
Storage bucket	<code>id 195596 · ticket attachments</code>
AI key (asset)	<code>SESAP_Anthropic_Key · server-side only, never in repo</code>

Baseline requirements — all met

Section 4 of the brief, the mandatory floor.

- ✓ **4.1 Coded App** — TypeScript Coded App; lists all active tickets live from Data Fabric; search & sort; full ticket detail view; create-ticket persists to Data Fabric; in-app actions start the orchestration.
- ✓ **4.2 Data Fabric** — a `Ticket` entity with ID, subject, description, priority, status, assignee, created date; `AssignmentStatus` is updated by the automation (the write-back).
- ✓ **4.3 RPA workflow** — a published robot doing meaningful work (AI triage + auto-assign + notify); reads and writes the Ticket entity; invoked by the orchestration, not run in isolation.
- ✓ **4.4 Maestro BPMN** — start event, multiple tasks, decision gateways, end events; models a recognisable ticket lifecycle; invokes the RPA workflow; shares Data Fabric state.
- ✓ **4.5 End-to-end** — the full loop works from an app action through Maestro and the robot to a Data Fabric write-back the app reflects; demonstrable live and on video.

Bonus items attempted

Nine tracks. Requested 30 → capped at the +25 maximum. Stated honestly.

Agentic / AI triage

- ✓ Claude categorises, prioritises, summarises and recommends a resolution — inside the robot, within the flow. **+6**

Document IDP

- ✓ On-device OCR reads images / ID cards; pdf.js extracts PDF text — attachments are read out in the copilot. **+4**

Human-in-the-loop

- ✓ Procurement & internal-request tickets route to supervisor approval, surfaced in the app and by email. **+4**

SLA & escalation

- ✓ SLA timers per priority; overdue tickets auto-escalate to Urgent and alert the supervisor. **+3**

Role-based views

- ✓ Distinct supervisor, agent, and customer experiences from one codebase. **+3**

Analytics / KPIs

- ✓ Open by priority, average resolution time, SLA compliance, created-vs-resolved trend. **+3**

Engineering quality

- ✓ Automated tests, reusable components, and error handling that degrades gracefully instead of crashing. **+3**

Notifications

- ✓ Real Gmail emails on every lifecycle event, CC to the supervisor. **+2**

Polished UX

- ✓ On-brand banking interface a support lead would use daily. **+2**

Honest note: Human-in-the-loop approval is implemented as an app-and-email approval flow (matching the brief's "with the app or email surfacing it"), not a Maestro Action Center user task. Everything else is exactly as described.

SESAP · Stanbic IBTC Enterprise Support & Automation Platform · Coded App + RPA +
Maestro + Data Fabric · org stanbgdgzsb / DefaultTenant